FIT2109 Computer Science Workshop

Week 1: Introduction to the Shell

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

Why FIT2109 starts with the shell

The shell is not here to replace the GUI

GUIs are excellent for visible, one-off, standard tasks.

The shell is the layer underneath modern software work

When setup fails, tools need configuration, logs need inspection, a server is remote, or a task must be repeated, the shell becomes the common control surface.

Why CS students must learn the shell

GUIs abstract away the system

- Hide how software actually works,
- Tied to one OS or interface,
- Cannot be scripted or automated,
- Fail when facing the unfamiliar.

The shell is foundational

- Direct access to system concepts,
- Works everywhere (local, remote, cloud, containers),
- Every task is scriptable and repeatable,
- The tool for debugging and fixing failures.

The shell connects the whole unit

Shell $\rightarrow$ scripting $\rightarrow$ Git $\rightarrow$ remote machines $\rightarrow$ containers

$\rightarrow$ debugging $\rightarrow$ profiling $\rightarrow$ automation $\rightarrow$ agentic coding

Week 1 goal

Build practical understanding of commands, files, paths, permissions, and pipelines.

Learning goals

By the end of this workshop, you should be able to:

- Explain what the shell prompt, working directory, and pathnames mean.
- Describe the filesystem as a tree of directories and files.
- Use core commands to inspect and change the filesystem.
- Distinguish a stored program from a running process.
- Interpret simple permission strings and explain why a script may not run.
- Explain `$PATH`, standard streams, redirection, pipes, and exit status.

The shell interprets commands

Key idea

The shell reads a line of text, interprets it, runs a command, and reports the result.

- A command has a **name**, optional **options**, and optional **arguments**.
- The shell always has context: user, machine, current directory, environment.
- The output is often more important than the command itself.

Typical shape

```
command -option argument
```

Command anatomy: name, options, arguments

Read the command from left to right

```
ls -l scripts
```

Command name

`ls`

The action to run: list directory contents.

Option

`-l`

Changes how the command behaves: use long format.

Argument

`scripts`

The thing the command acts on: this directory.

Hint

Use the command `man ls` to learn all options for `ls`.

Commands are abbreviations

The name usually tells you what the command does

When you see an unfamiliar command, its full name is often the best definition.

- `pwd` — **p**rint **w**orking **d**irectory
- `ls` — **l**ist
- `cd` — **c**hange **d**irectory
- `mkdir` — **m**ake **dir**ectory
- `cp` — **c**o**p**y
- `mv` — **m**o**v**e
- `rm` — **r**e**m**ove
- `cat` — con**cat**enate
- `chmod` — **ch**ange **mod**e
- `ps` — **p**rocess **s**tatus
- `man` — **man**ual
- `grep` — **g**lobal **r**egular **e**xpression **p**rint
- `wc` — **w**ord **c**ount

Your location changes what commands mean

Key idea

The shell is always "standing" in one directory. Relative pathnames are interpreted from there.

- `pwd` shows the current working directory.
- `cd` changes the current working directory.
- `ls` lists files in a directory.
- Many command mistakes are really location mistakes.

Files and directories form a tree

Key idea

Directories contain files and other directories. A pathname describes a route through this tree.

- The root of the tree is written as `/`.
- Your home directory is your usual personal starting point.
- A directory can have children and a parent.
- Commands such as `ls`, `cd`, and `pwd` help you inspect your place in the tree.

Paths tell the shell where to look

Absolute path

Starts from the root of the filesystem.

`/Users/student/fit2109/week1`

Relative path

Starts from the current working directory.

`data/raw`

- `.` means current directory.
- `..` means parent directory.
- `~` means your home directory.

Demo 1: shell context

Watch

Before changing anything, identify the user, machine, current directory, interactive shell, and visible files.

Note

The shell always has context. The same command can mean something different after the context changes.

Demo 2: move through the filesystem

Watch

After each directory change, check where the shell is standing.

Note

`.`, `..`, `~`, absolute paths, and relative paths are all ways to describe location.

Handout Activity 1: shell context and location

Now work on Activity 1 in the handout

Decode shell context, identify command anatomy, predict `cd` destinations, and explain why location mistakes happen.

Group output

Complete Parts A–D: context table, command anatomy, destination prediction, and two-sentence explanation.

Commands inspect or change filesystem state

Key idea

Files and directories are state. Commands either inspect state or change state.

- `mkdir`: create directories
- `touch`: create/update a file
- `cp`: copy; the original stays
- `mv`: move or rename; the old path disappears
- `rm`: remove
- `ls`: inspect

Professional habit

Inspect before and after a command when you are learning or when the change is risky.

Permissions decide who can do what

Key idea

A file may exist, but the system still checks whether you are allowed to read, write, or execute it.

How to read the permission string

`- rwx r-x r-x` = filetype | owner | group | others

`r`: read `w`: write `x`: execute

- A `-` in place of a letter means that permission is absent.
- The first character shows file type: `-` for regular file, `d` for directory.
- `chmod u+x file` adds execute permission for the owner.

Demo 3: filesystem changes

Watch

After each file operation, ask: what exists now, and which path changed?

Note

`cp` leaves the original behind. `mv` makes the old path disappear. `rm` removes an item.

Filename patterns are expanded by the shell

Key idea

Patterns such as `*.txt` are expanded by the shell into matching filenames before the command runs.

- `ls *.txt` lists files whose names end in `.txt`.
- `ls *.sh` lists shell-script filenames in the current directory.
- If there is no match, different shells may report or pass the pattern differently.

For now

Use this as a preview. Full coverage of patterns, quoting, and expansion is in Week 2.

Demo 4: make a script executable

Watch

The script file exists before it can run. The permission string explains why.

Note

`chmod u+x` changes the owner's execute permission. It does not change the script contents.

Handout Activity 2: filesystem state, patterns, and permissions

Now work on Activity 2 in the handout

Trace filesystem state, draw the final tree, predict filename-pattern expansion, and diagnose permission changes.

Group output

Complete Parts A–D: state trace, final tree, pattern table, and `chmod u+x` explanation.

Programs become processes when they run

Key idea

A program is stored code. A process is a running instance of a program.

- Running a command often starts a process.
- A process has an identifier called a PID.
- Some commands finish quickly; others keep running.
- Error messages often come from the program/process, not from the shell itself.

$PATH is where the shell searches

Key idea

When you type a command name, the shell does not search the whole computer. It searches directories listed in `$PATH`.

- Some commands are built into the shell, such as `cd`.
- Some commands are external programs, such as `ls`.
- If you provide a path, such as `./scripts/run.sh`, the shell uses that path directly.

Demo 5: processes

Watch

Identify the current shell process, then compare shell builtins, external commands, and commands found through `$PATH`.

Note

`$$` expands to the PID of the current shell. `pid`, `ppid`, and `comm` describe running processes.

Demo 6: command lookup and explicit paths

Watch

Compare asking the shell to find `run.sh` with giving the shell the explicit path `./scripts/run.sh`.

Pause and reason

Why might `run.sh` fail even when the file exists?

Handout Activity 3: execution, processes, and $PATH

Now work on Activity 3 in the handout

Classify programs and processes, read `ps` output, explain command lookup, and connect Bash/Zsh to scripts.

Group output

Complete Parts A–D, especially why `run.sh` fails but `./scripts/run.sh` works.

Bash, Zsh, and what changes later

This course uses Bash

Scripts and examples use `#!/usr/bin/env bash`. Bash is the default on most Linux systems and WSL.

macOS defaults to Zsh since macOS Catalina (2019)

You may see a `%` prompt instead of `$`. That is normal — you are in Zsh.

- For all Week 1 commands (`ls`, `cd`, `pwd`, pipes, redirection), **Bash and Zsh behave identically**.
- Differences appear in scripting syntax and some expansions — covered in Week 2.
- A script with `#!/usr/bin/env bash` always runs in Bash, regardless of your interactive shell.
- `echo "$SHELL"` tells you which shell is running interactively.

Streams separate input, output, and errors

Key idea

Programs usually read input and produce output. The shell can redirect where those streams go.

- **Standard input**: where a program reads from by default.
- **Standard output**: where normal results go by default.
- **Standard error**: where error messages go by default.
- >, », <, and 2> redirect streams.

Demo 7: redirect output and errors

Watch

Track which text appears on screen, which text is saved to a file, and which stream carries an error message.

Note

Standard output and standard error are separate streams. Redirection changes where each stream goes.

Pipes make small tools work together

- A pipe sends one program's output into another program's input.
- Each command can stay small and focused.
- A pipeline becomes a custom workflow.
- Pipeline order matters because each step transforms the stream.

Key idea

```
program A | program B | program C
```

Demo 8: answer a question from a log

Watch

Start with a log file, filter the relevant lines, count them, then summarize repeated errors.

Note

A pipeline is a chain of transformations. Each command should make the next command's job easier.

Exit status records success or failure

Key idea

After a command runs, it returns a small number called an exit status. In Unix-style shells, `0` usually means success and nonzero usually means failure.

- `command not found`: the shell could not locate the command.
- `permission denied`: the file exists, but access is not allowed.
- `no such file or directory`: the path does not point to an existing item.
- Pipelines have special status behaviour; `set -o pipefail` changes this in scripts.

Error messages are clues, not noise

Key idea

Every error message is a clue. Read it before doing anything else.

1. **Read the error message** — it usually says what failed and why.
2. **Check the manual** — `man <command>` or `<command> --help`.
3. **Search online** — paste the exact error text plus the command name.
4. **Ask AI** — include what you ran, expected, and got.

Demo 9: inspect success and failure

Watch

After each command, compare what appeared on screen with the exit status recorded by the shell.

Note

Output and exit status are related, but they are not the same thing. A pipeline has its own status behaviour.

Demo 9 continued: from error to answer

Watch

Use the exact error text to decide the next diagnostic step.

Note

- **Online**: search the exact error text plus the command name.
- **AI**: "I ran `ls missing.txt`. I expected a file listing. I got: `ls: cannot access 'missing.txt': No such file or directory`. What could cause this?"

Handout Activity 4: streams, pipelines, and failure clues

Now work on Activity 4 in the handout

Map streams, build a log-analysis pipeline, explain each stage, inspect exit status, and turn an error into a next step.

Group output

Complete Parts A–E: stream mapping, pipeline, stage explanation, exit-status prediction, and error diagnosis.

Final checkpoint: Poll Everywhere

pollev.com/fit2109

Join today's checkpoint

Scan the QR code or go to pollev.com/fit2109.

Purpose

Use the Poll Everywhere questions to check which shell ideas need one more example before we finish.

What should feel different after today?

- You can ask: **where am I?**
- You can inspect: **what files exist and what permissions do they have?**
- You can recognise: **which process or command produced this result?**
- You can explain: **why did this command run or fail?**
- You can combine tools: **how can I answer this question from text?**
- You can unblock yourself: **what to do when a command fails.**

The bigger idea

The shell turns computing actions into visible, repeatable, explainable steps.

Exit ticket

1. What does `pwd` tell you?
2. What is one difference between `cp` and `mv`?
3. Why might `./scripts/run.sh` work when `run.sh` does not?
4. What does a pipe connect?
5. What does exit status `0` usually mean?
6. Name two ways to look up what an unfamiliar flag does.

Reflection: Which shell idea do you want to practise more this week?

Q&A and next steps

Questions?

Before next time

Practise the Week 1 applied tasks until you can explain both the command and the output.